Bodega Simeon

Nombre: Rafael Vega Librero
Fecha: 14/12/2025
Clase: 1ºDAW

ÍNDICE

1. Introducción proyecto.....2
2. Modelo Entidad Relación.....2
3. Modelo Relacional en WorkBench.....3
4. Carga de Proyecto en AWS..... 3
5. Carga Masiva en Tablas.....4
6. Consultas..... 5
7. VISTAS.....8
8. Funciones y Procedimientos.....9
8.1 Funciones..... 9
8.2 PROCEDIMIENTOS.....10
9. Triggers.....11
10. GitHub.....12
11. Conclusión..... 13

1. Introducción proyecto

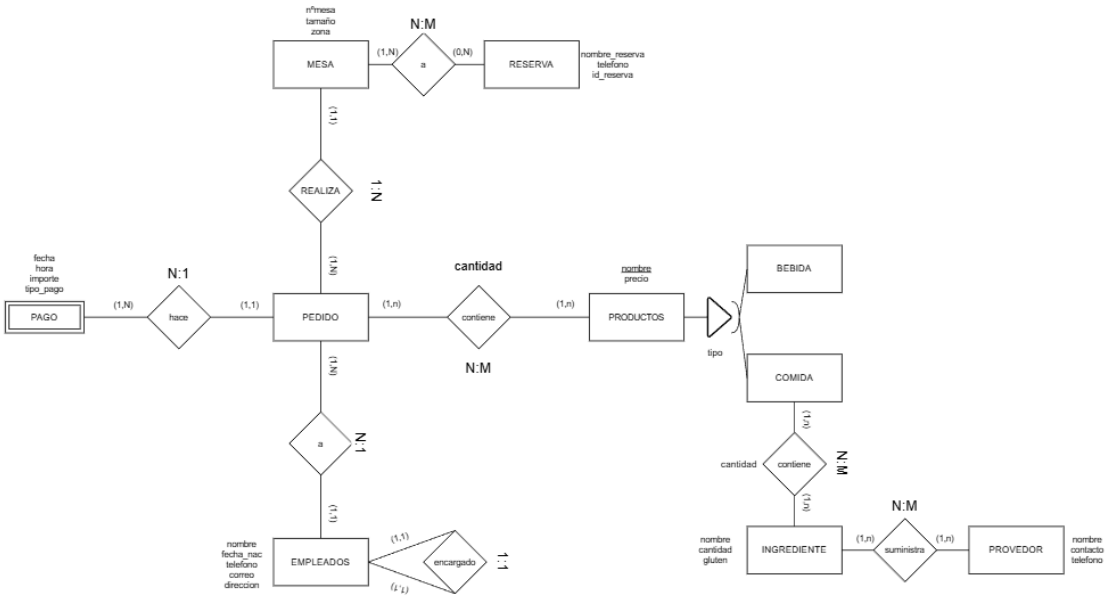
He decidido basar mi proyecto de base de datos sobre el local en el que trabajo “Bodega Simeón”, Es un bar familiar que se encuentra en Bormujos.

He decidido escogerlo porque me parece que es un buen proyecto para aprender, ya que tengo que saber las relaciones que se hacen entre las diversas tablas que tiene que contener un bar.

Ya que tienen que tener unas tablas con Empleados, Producto, Pedido, Mesas, etc…

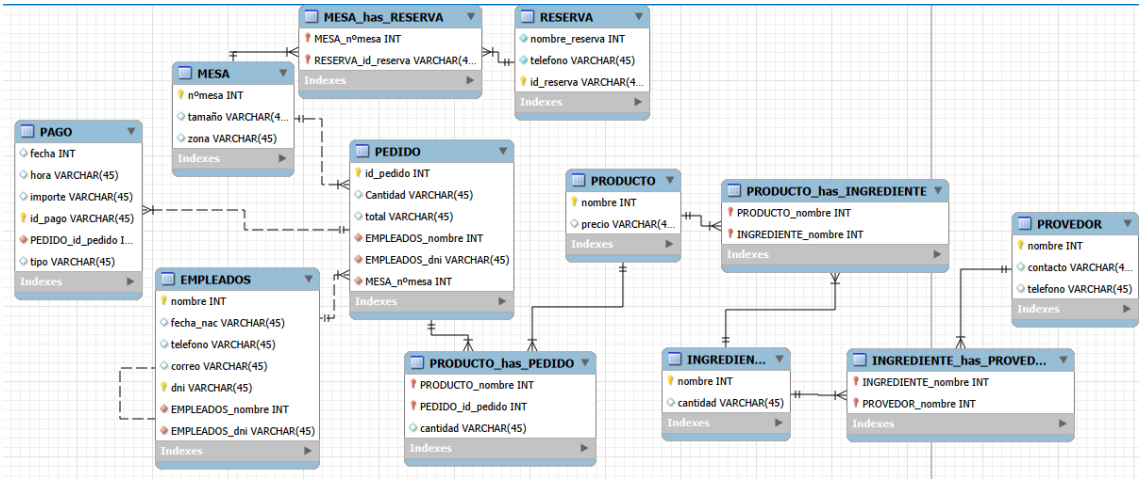
2. Modelo Entidad Relación

En este modelo podemos ver como vamos a organizar nuestro bar “Bodega Simeón”, Como se ve en la imagen tenemos varias entidades que son las siguientes (MESA, RESERVA, PAGO, PEDIDO, EMPLEADOS, PRODUCTOS, etc…)



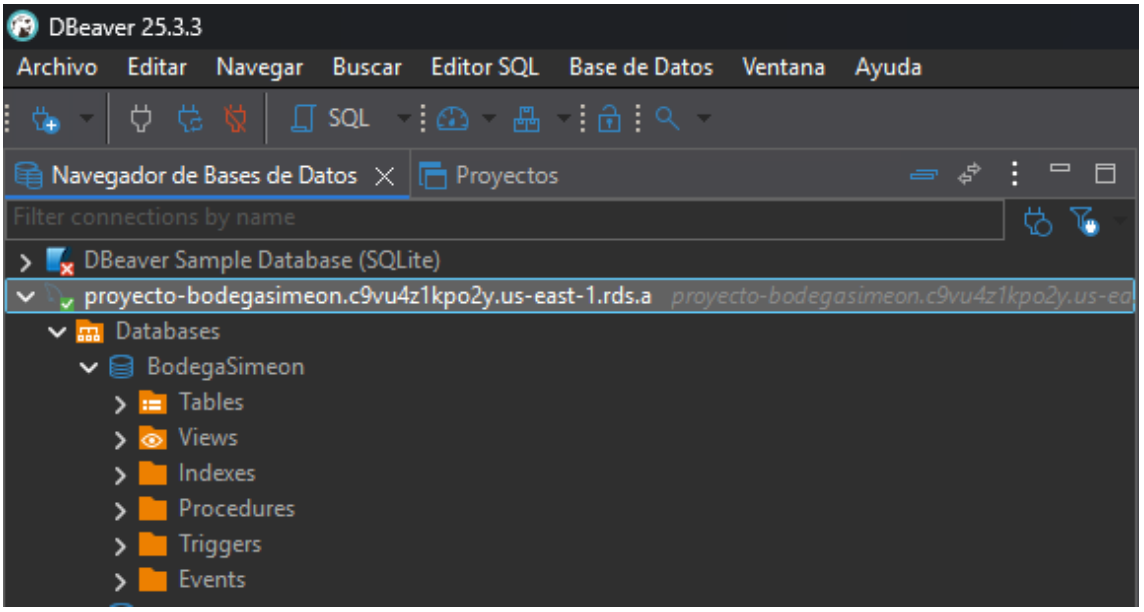
3. Modelo Relacional en WorkBench

Aqui podemos ver el modelo Relacional, donde relacionamos las tablas con sus PK y así obtienen algunas las FK.



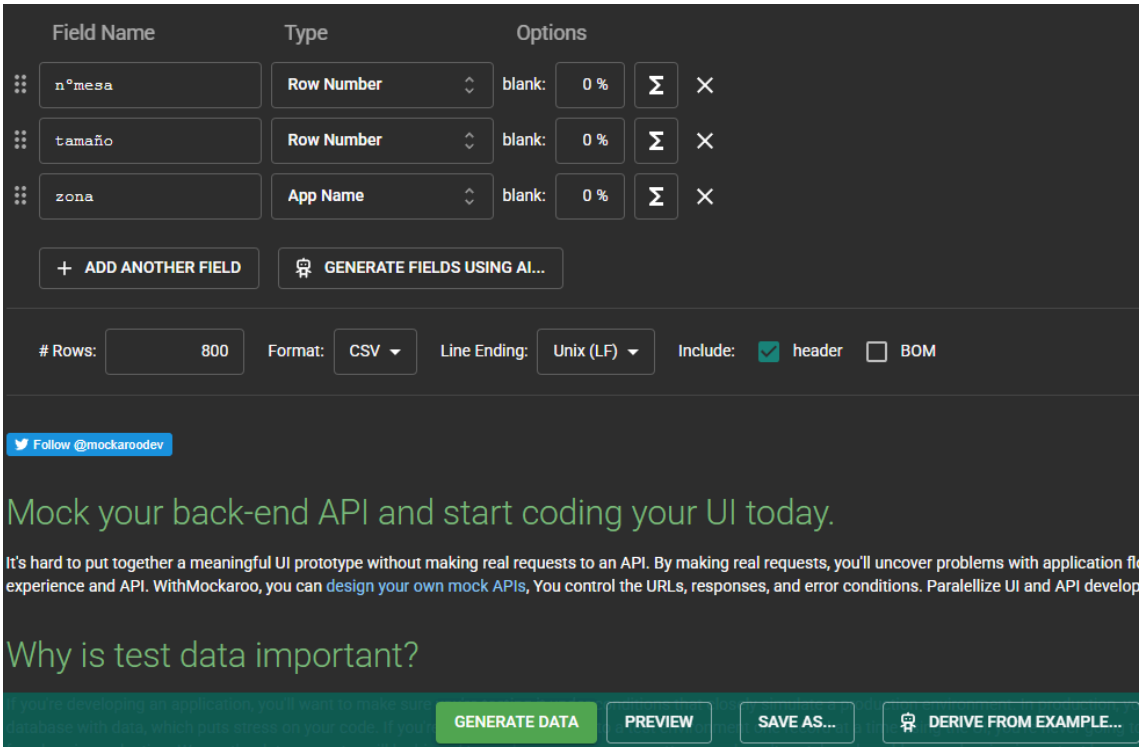
4. Carga de Proyecto en AWS

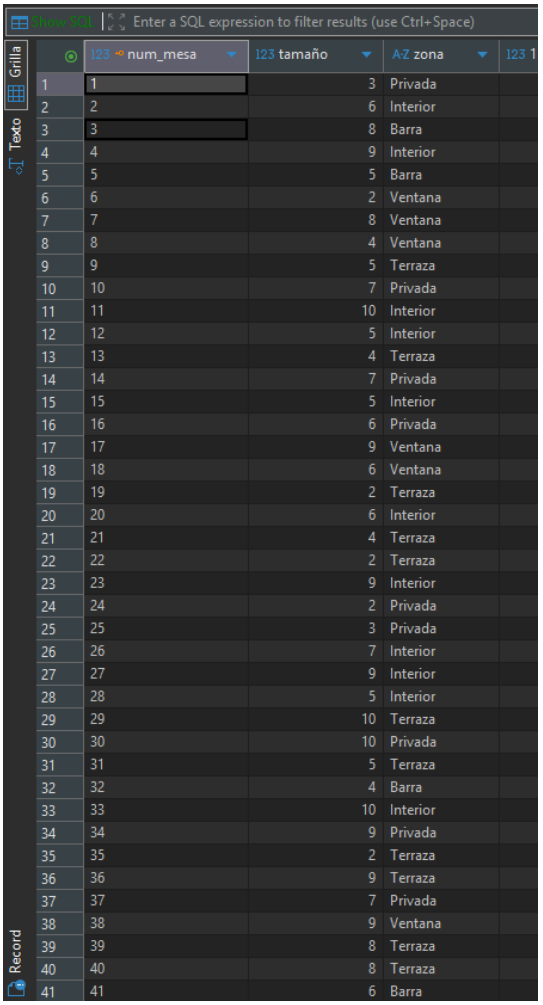
Para cargar los datos en AWS debemos exportar, el SQL de nuestro workbench y nos dará un archivo, el cual ejecutaremos en AWS y se nos creará nuestra base de datos.



5. Carga Masiva en Tablas

Usamos alguna página web que se especializa en generar datos aleatorios y reales para nuestras bases de datos. Yo en este caso he usado “Mockaroo” ya que me parecía la mejor.





Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	num_mesa	tamaño	zona
1	1	3	Privada
2	2	6	Interior
3	3	8	Barra
4	4	9	Interior
5	5	5	Barra
6	6	2	Ventana
7	7	8	Ventana
8	8	4	Ventana
9	9	5	Terraza
10	10	7	Privada
11	11	10	Interior
12	12	5	Interior
13	13	4	Terraza
14	14	7	Privada
15	15	5	Interior
16	16	6	Privada
17	17	9	Ventana
18	18	6	Ventana
19	19	2	Terraza
20	20	6	Interior
21	21	4	Terraza
22	22	2	Terraza
23	23	9	Interior
24	24	2	Privada
25	25	3	Privada
26	26	7	Interior
27	27	9	Interior
28	28	5	Interior
29	29	10	Terraza
30	30	10	Privada
31	31	5	Terraza
32	32	4	Barra
33	33	10	Interior
34	34	9	Privada
35	35	2	Terraza
36	36	9	Terraza
37	37	7	Privada
38	38	9	Ventana
39	39	8	Terraza
40	40	8	Terraza
41	41	6	Barra

6. Consultas

1. Mostrar el total gastado por cada mesa, indicando también cuántos pedidos se han hecho en cada una, ordenado de mayor a menor gasto.

```
select
  m.num_mesa,
  m.zona,
  count(p.id_pedido) as num_pedidos,
  round(sum(p.total), 2) as total_gastado
from mesa m
join pedido p on m.num_mesa = p.num_mesa
group by m.num_mesa, m.zona
order by total_gastado desc;
```

	num_mesa	zona	num_pedidos	total_gastado
1	3	interior	1	0
2	2	interior	1	0
3	4	patio	1	0

2. Mostrar los productos más vendidos, indicando cuántas unidades se han vendido y cuánto dinero han generado.

```
select
  pr.nombre as producto,
  pr.categoria,
  sum(dp.cantidad) as unidades_vendidas,
  round(sum(dp.subtotal), 2) as dinero_generado
from producto pr
join detalle_pedido dp on pr.id_producto = dp.id_producto
group by pr.id_producto, pr.nombre, pr.categoria
order by unidades_vendidas desc, dinero_generado desc;
```

	producto	categoria	unidades_vendidas	dinero_generado
1	ensaladilla	tapa	2	
2	mosto de la casa	bebida	2	
3	coca cola	bebida	2	
4	vino tinto de la casa	bebida	2	
5	arroz de carrillada	arroz	1	
6	migas con huevo	plato	1	
7	carnes a la brasa	carne	1	

3. Mostrar los empleados que han atendido pedidos cuyo total está por encima del total medio de todos los pedidos.

```
select
  e.nombre as empleado,
  p.id_pedido,
  p.total
from empleado e
join pedido p on e.id_empleado = p.id_empleado
where p.total > (
  select avg(total)
  from pedido
)
order by p.total desc;
```

empleado(+) 1

select e.nombre as emplea | Enter a SQL expression to filter results (use C

Grilla | Texto

empleado	id_pedido	total

4. Mostrar los ingredientes que se usan en más de un producto, indicando en cuántos productos aparece cada uno.

```
select
  i.nombre as ingrediente,
  count(pi.id_producto) as veces_usado
from ingrediente i
join producto_ingrediente pi on i.id_ingrediente = pi.id_ingrediente
group by i.id_ingrediente, i.nombre
having count(pi.id_producto) > 1
order by veces_usado desc, ingrediente asc;
```

	ingrediente	veces_usado
1	aceite de oliva	9
2	arroz	4
3	alcachofas	2
4	huevo	2
5	langostinos	2
6	patatas	2

5. mostrar los pedidos con su empleado, mesa y método de pago, incluyendo sólo aquellos pedidos que ya han sido pagados.

```
select
  p.id_pedido,
  e.nombre as empleado,
  m.num_mesa,
  m.zona,
  p.total,
  pa.metodo_pago,
  pa.fecha as fecha_pago
from pedido p
join empleado e on p.id_empleado = e.id_empleado
join mesa m on p.num_mesa = m.num_mesa
join pago pa on p.id_pedido = pa.id_pedido
where pa.estado = 'pagado'
order by pa.fecha desc;
```

	id_pedido	empleado	num_mesa	zona	total	metodo_pago	fecha_pago
1	3	maría lópez	4	patio	0	tarjeta	2026-03-19 15:

7. VISTAS

```sql
-- PRODUCTOS MAS VENDIDOS
create or replace view vw_productos_mas_vendidos as
select
    pr.nombre as producto,
    pr.categoria,
    sum(dp.cantidad) as unidades_vendidas,
    round(sum(dp.subtotal), 2) as dinero_generado
from producto pr
join detalle_pedido dp on pr.id_producto = dp.id_producto
group by pr.id_producto, pr.nombre, pr.categoria;

select * from vw_productos_mas_vendidos;
```

producto(+) 1

Enter a SQL expression to filter results (use Ctrl+Space)

	producto	categoria	unidades_vendidas	dinero_generado
1	ensaladilla	tapa	2	0
2	mosto de la casa	bebida	2	0
3	arroz de carrillada	arroz	1	0
4	migas con huevo	plato	1	0
5	coca cola	bebida	2	0
6	carnes a la brasa	carne	1	0
7	vino tinto de la casa	bebida	2	0

```sql
-- PEDIDOS PAGADOS COMPLETOS
create or replace view vw_pedidos_pagados as
select
    p.id_pedido,
    e.nombre as empleado,
    m.num_mesa,
    m.zona,
    p.total,
    pa.metodo_pago,
    pa.fecha as fecha_pago
from pedido p
join empleado e on p.id_empleado = e.id_empleado
join mesa m on p.num_mesa = m.num_mesa
join pago pa on p.id_pedido = pa.id_pedido
where pa.estado = 'pagado';

select * from vw_pedidos_pagados;
```

vw_pedidos_pagados 1

Enter a SQL expression to filter results (use Ctrl+Space)

	id_pedido	empleado	num_mesa	zona	total	metodo_pago	fecha_pago
1	3	maría lópez	4	patio	0	tarjeta	2026-03-19 15:40:0

8. Funciones y Procedimientos

8.1 Funciones

```
-- 1. devolver el número total de pedidos que ha atendido un empleado.

delimiter //

create function fn_total_pedidos_empleado(p_id_empleado int)
returns int
deterministic
begin
    declare total_pedidos int;

    select count(*)
    into total_pedidos
    from pedido
    where id_empleado = p_id_empleado;

    return total_pedidos;
end //

delimiter ;

select fn_total_pedidos_empleado(1) as total_pedidos;
```

```
-- 2. devolver el importe medio de los pedidos de una mesa.

delimiter //

create function fn_media_gasto_mesa(p_num_mesa int)
returns decimal(10,2)
deterministic
begin
    declare media_gasto decimal(10,2);

    select ifnull(avg(total), 0)
    into media_gasto
    from pedido
    where num_mesa = p_num_mesa;

    return media_gasto;
end //

delimiter ;

select fn_media_gasto_mesa(3) as media_gasto;
```

8.2 PROCEDIMIENTOS

```
-- 1. mostrar todos los pedidos de una mesa concreta.

delimiter //

create procedure sp_pedidos_por_mesa(in p_num_mesa int)
begin
    select
        id_pedido,
        fecha,
        total,
        estado
    from pedido
    where num_mesa = p_num_mesa
    order by fecha desc;
end //

delimiter ;

call sp_pedidos_por_mesa(3);
```

```
-- 2. actualizar el estado de una mesa.

delimiter //

create procedure sp_actualizar_estado_mesa(
    in p_num_mesa int,
    in p_estado varchar(20)
)
begin
    update mesa
    set estado = p_estado
    where num_mesa = p_num_mesa;
end //

delimiter ;

call sp_actualizar_estado_mesa(2, 'ocupada');
```

```
-- 3. mostrar un resumen de un empleado indicando su nombre y cuántos pedidos ha atendido. este procedimiento usa una función.

delimiter //

create procedure sp_resumen_empleado(in p_id_empleado int)
begin
    select
        nombre,
        fn_total_pedidos_empleado(p_id_empleado) as pedidos_atendidos
    from empleado
    where id_empleado = p_id_empleado;
end //

delimiter ;

call sp_resumen_empleado(1);
```

9. Triggers

```
-- 1. antes de insertar una línea en detalle_pedido, calcular automáticamente el subtotal

delimiter //

create trigger trg_calcular_subtotal_before_insert
before insert on detalle_pedido
for each row
begin
    set new.subtotal = new.cantidad * new.precio_unitario;
end //

delimiter ;

select * from detalle_pedido;

insert into detalle_pedido (id_pedido, id_producto, cantidad, precio_unitario, subtotal)
values (1, 13, 3, 2.30, 0);

select *
from detalle_pedido
where id_pedido = 1 and id_producto = 13
order by id_detalle desc;
```

detalle_pedido 1

Enter a SQL expression to filter results (use Ctrl+Space)

	id_detalle	id_pedido	id_producto	cantidad	precio_unitario	subtotal
1	8	1	13	3	2,3	6,9

```
-- 2. después de insertar un pago, cambiar automáticamente el estado del pedido a cobrado.

delimiter //

create trigger trg_marcar_pedido_cobrado_after_insert
after insert on pago
for each row
begin
    update pedido
    set estado = 'cobrado'
    where id_pedido = new.id_pedido;
end //

delimiter ;

select id_pedido, estado
from pedido
where id_pedido = 2;

insert into pago (id_pedido, fecha, importe, metodo_pago, estado)
values (2, now(), 14.10, 'bizum', 'pagado');

select id_pedido, estado
from pedido
where id_pedido = 2;
```

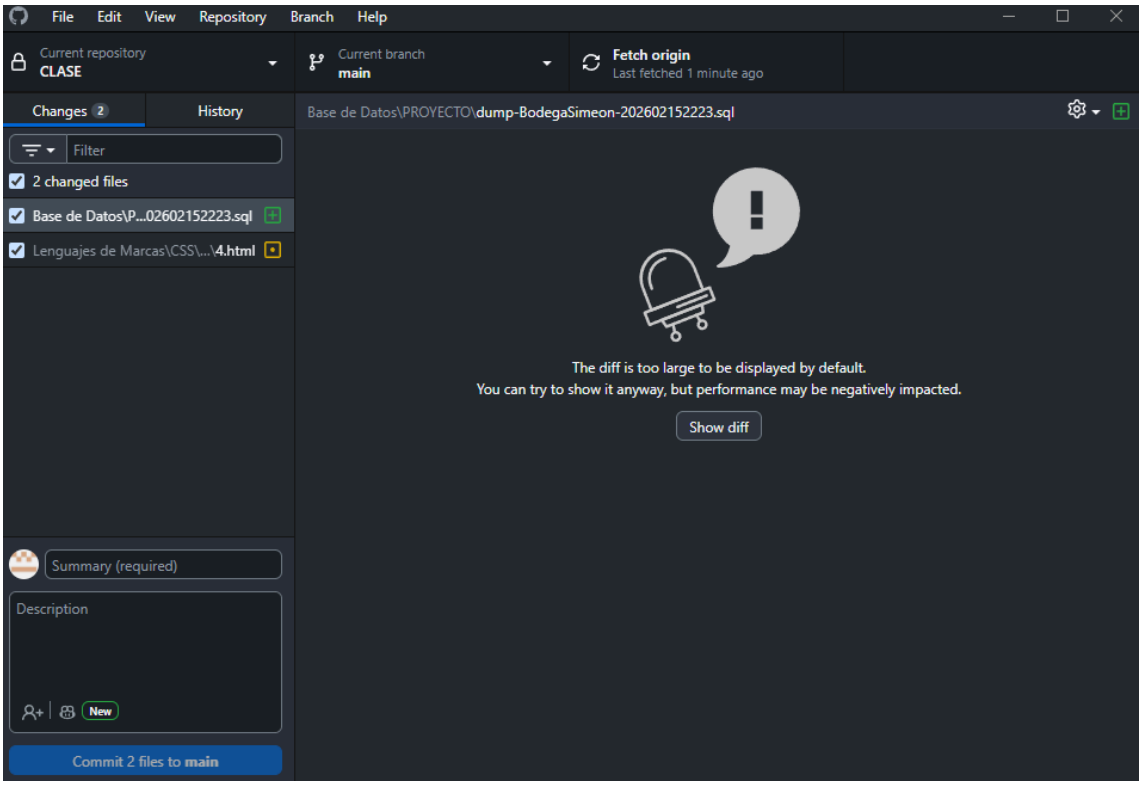
pedido 1

Enter a SQL expression to filter results (use Ctrl+Space)

	id_pedido	estado
1	2	cobrado

10. GitHub

Yo subo las cosas a GitHub por la aplicación de Desktop Github. Como se muestra en la imagen esta aplicación detecta cambios en tu carpeta local de github en la que tu puedes darle un nombre y subirlo a tu repositorio online. Ya que estos están conectados.



Para acceder al GIT, podéis acceder pinchando AQUÍ

11. Conclusión

Durante el desarrollo de la base de datos he tenido varias dificultades, especialmente relacionadas con la carga de datos y las claves foráneas. Muchos errores surgieron por inconsistencias en los nombres de columnas (como diferencias entre n°mesa, núm_mesa, etc.) y por restricciones de claves foráneas que impedían insertar registros cuando no coincidían exactamente los tipos o valores. Esto me obligó a revisar la estructura de las tablas, los tamaños de los campos y el mapeo correcto en la importación de los archivos CSV. Aunque el proceso fue más complejo de lo previsto, me permitió entender mejor cómo funcionan las relaciones entre tablas y la importancia de la coherencia en el diseño de una base de datos relacional